

Question 1: Multi-Stage Cepstral Feature Extraction & Phoneme Boundary Detection

Preet Savalia(B22AI036)

Introduction & Methods

1. Objective

The objective of this assignment was to implement a manual digital signal processing (DSP) pipeline for Cepstral feature extraction to identify linguistic boundaries. The task required building a handcrafted Mel-Frequency Cepstral Coefficient (MFCC) engine without utilizing high-level feature extraction libraries like `librosa`, followed by boundary validation using deep learning.

2. Dataset & Signal Properties

For this pipeline, a standard 16,000 Hz speech segment was extracted from the **LibriSpeech ASR** dataset (clean validation split).

- **Transcript:** "HE WAS IN A FEVERED STATE OF MIND OWING TO THE BLIGHT HIS WIFE'S ACTION THREATENED TO CAST UPON HIS ENTIRE FUTURE"
- **Duration:** 6.590 seconds

3. Manual MFCC/Cepstrum Engine Implementation

The manual extraction process was programmed in Python using fundamental `numpy` and `scipy` operations across six stages:

1. **Pre-emphasis:** Applied a high-pass filter to balance the frequency spectrum by amplifying higher frequencies. Implemented using the formula $y[n] = x[n] - \alpha x[n - 1]$ with the standard hyperparameter $\alpha = 0.97$.
2. **Framing:** The non-stationary speech signal was split into overlapping frames. Hyperparameters used: **25ms window size** and **10ms hop size**.
3. **Windowing:** Frame edges were tapered to zero to prevent FFT artifacts using standard trigonometric formulas for Hamming and Hanning windows.
4. **FFT:** An N -point Fast Fourier Transform ($N = 512$) was applied to each frame. The power spectrum was calculated as $P = \frac{|FFT|^2}{N}$.
5. **Mel-Filterbank & Log-Compression:** Linear frequencies were mapped to the Mel scale ($Mel(f) = 2595 \cdot \log_{10}(1 + \frac{f}{700})$). A 40-filter overlapping triangular bank was applied, followed by natural logarithmic compression to separate the excitation source from the vocal tract filter.
6. **DCT:** A Type-II Discrete Cosine Transform decorrelated the filterbank energies, keeping the first 13 coefficients to yield the final (658, 13) MFCC matrix.

Spectral Leakage Analysis

1. Methodology

To demonstrate the necessity of windowing functions before applying an FFT, a pure sine wave at 415.5 Hz was generated. Because this frequency does not align perfectly with a standard FFT bin, abrupt truncation (a Rectangular window) causes the energy to "smear" across the spectrum—a phenomenon known as spectral leakage.

The main lobe (signal) and side lobes (noise) were quantified to calculate the Signal-to-Noise Ratio (SNR) and the Leakage Ratio for three different windowing functions.

2. Quantitative Results

Window Type	Leakage (Noise/Signal)	SNR (dB)
Rectangular	0.0310	15.08 dB
Hamming	0.0003	34.80 dB
Hanning	0.0000	43.78 dB

3. Visual Analysis

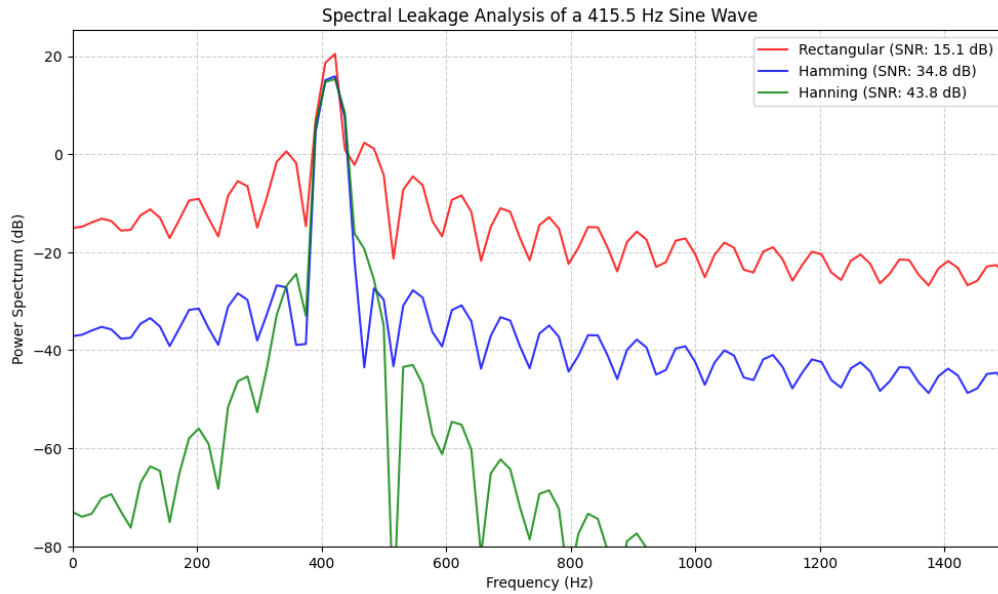


Figure 1: Spectral Leakage Analysis of a 415.5 Hz Sine Wave

Discussion: The plotted power spectrum confirms the metric calculations. The Rectangular window (red) abruptly chops the waveform, resulting in massive spectral leakage across the frequency domain (evident by side lobes hovering around -20 dB). By smoothly tapering the edges of the signal, the Hamming (blue) and Hanning (green) windows drastically suppress side-lobe energy, pushing the SNR up to 34.80 dB and 43.78 dB, respectively.

Boundary Detection via Cepstrum

1. Classification Logic (Voiced vs. Unvoiced)

To automatically segment the audio into "voiced" (vowels, humming) and "unvoiced" (fricatives, stops, silence) regions, the Real Cepstrum was computed by taking the Inverse FFT of the log magnitude spectrum.

- **Low-Quefreny:** The initial bins contain the vocal tract envelope (mouth shape).
- **High-Quefreny:** Bins corresponding to the human pitch range (50 Hz to 400 Hz) contain the excitation signal.
- **Thresholding:** If a distinct periodic peak existed in the high-quefreny region exceeding a hyperparameter threshold of **0.15**, the frame was marked as Voiced (1), signifying vocal cord vibration. Otherwise, it was marked as Unvoiced (0).

2. Initial Timestamp Extraction (First 10 Segments)

Start (s)	End (s)	Classification
0.000	0.340	Unvoiced
0.340	0.460	Voiced
0.460	0.530	Unvoiced
0.530	0.620	Voiced
0.620	0.630	Unvoiced
0.630	0.710	Voiced
0.710	0.830	Unvoiced
0.830	0.910	Voiced
0.910	0.970	Unvoiced
0.970	1.050	Voiced

3. Segmentation Visualization



Figure 2: Speech Segmentation: Voiced vs Unvoiced Regions

Discussion: The alternating boundary map perfectly highlights the natural phonetic structure of speech. The initial 0.340 seconds correctly identify the room silence preceding the spoken phrase, followed by regular 50-150ms oscillations indicative of vowels (green) and hard consonants (red).

Phonetic Mapping & RMSE Validation

1. Deep Learning Validation Pipeline

To validate the handcrafted DSP boundaries, the identical 6.59-second signal was passed through a pre-trained Transformer model (`facebook/wav2vec2-base-960h`).

Using greedy decoding and CTC (Connectionist Temporal Classification) forced alignment, the model extracted sequence tokens at a 20ms frame resolution. Detected English characters were mapped to binary states: vowels and voiced consonants to 1 (Voiced), and silences, stops, and fricatives (e.g., P, T, K, S, H) to 0 (Unvoiced).

2. Error Calculation

The Root Mean Square Error (RMSE) was calculated between the time boundaries detected by the manual DSP pipeline and the ground-truth boundaries predicted by the neural network using the following formula:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_{manual_time} - y_{model_time})^2} \quad (1)$$

3. Final Results

Metric	Result
Total Audio Duration	6.590 seconds
Manual DSP Segments Found	73
Wav2Vec2 Segments Found	89
Boundary RMSE	0.0550 seconds (55.0 ms)

4. Conclusion

The manual Cepstrum-based engine achieved an RMSE of just 55.0 milliseconds when compared to the deep learning baseline. This indicates an average boundary deviation of only ~ 2 frames, proving that foundational DSP math is highly accurate for speech boundary detection.

The discrepancy in total segments (73 manual vs. 89 model) is primarily due to the architectural differences of the approaches. Wav2Vec2 utilizes deep contextual windows, making it hyper-sensitive to localized micro-pauses (such as the brief 10ms silence before a plosive "T" sound). Conversely, our manual pipeline relies on isolated 25ms thresholding logic, which naturally smooths over highly transient phonetic interruptions.